Assignment 07: Recursion

CSci 233: Application Program Development w/ Python

Objectives

- Get experience writing basic recursive functions in Python
- Learn about how to specify base cases to end recursive calls in a recursive function
- Learn about recursive case(s) and writing them for recursive Python function.s
- Get some experience with more advanced examples of recursive functions using mathematical recurrence relations.

Description

This assignment consists of 3 to 4 questions that cover topics from chapter 12 (Recursion) of our course textbook.

A recursive function is simply a function that calls itself. As we talked about in class, this may seem like a strange idea when first encountered. But recursion can be a useful way to express an algorithmic solution to many kinds of problems, especially for typical data structures and tree like algorithms often encountered in computer science and software application design.

In this assignment you will develop and write several recursive functions in Python to solve problems, from simple to more complex and interesting tasks.

Assignment Prerequisites and Setup

Before performing any assignment in this class, you need to have the following tasks already completed.

1. You need to have `git` tools installed on your system so that you can successfully clone repositories and create commits.
2. You need to have a GitHub account created.
   - You need to have successfully created a ssh key so that you can authenticate with and push commits back to git.
3. You need a working Python 3 distribution installed on the system you will work on these assignments with, that you can run Python scripts and the Python IDLE interface within.

See our class Getting Started with Python and Git for Class Assignments for links on setting up these needed development environment tools and configuration.

Assignment Questions

Start by cloning the assignment repository that was created for you when you accepted the assignment. There are 3 mostly empty Python script files in the top level of this assignment named `q01.py` through `q03.py`.

You should answer each of the following questions by writing appropriate Python code to solve the stated task. Once you are satisfied with your answer, you should create a commit and push it back to your GitHub repository for grading. In general you should do each question as a separate commit (see Git Best Practices).

Each commit you push to GitHub will create a release that will be graded. Look at the `Releases` on the right hand side of your `Code` page to see a summary of your grade results. Also the instructor will give feedback for all assignments in your `Feedback` Pull request. See the `Feedback` pull request listed on GitHub on the `Pull requests` tab.

You can and should test your code locally before creating commits and submitting it to GitHub. As mentioned, since you are now writing functions for your assignments, we are now using the `pytest` unit testing module in your autograder and to run tests against your work. So for example, to run the tests for question 01, from the command line you can do:

```
$ python3 -m pytest -v -s test/test_q01.py
==================================== test session starts ====================================
platform linux -- Python 3.14.9, pytest-9.1.1, pluggy-1.7.0 -- /opt/base/bin/python3
cachedir: .pytest_cache
rootdir: /workspaces/assg07-solution
collected 3 items

test/test_q01.py::test_base_cases PASSED
test/test_q01.py::test_recursive_cases PASSED
test/test_q01.py::test_larger_values PASSED

==================================== 3 passed in 0.01s ======================================
```

There should be tests for each question in the `test` subdirectory, named `test_q01.py`, `test_q02.py`, etc. for our class assignments. These will be (some of) the same `pytest` tests that are run by the autograder when you submit your work.

For the applications that your write in questions for an assignment, we are also still using the simple input / output diff comparison to test. These are invoked as part of the `pytest` tests of these questions. But as usual, when you are writing the applications, you can look for the expected output in the `test/data` subdirectory, to see exactly what your prompts and output should be formatted to look like in order to pass your tests. For example, if you run the tests for a question 03 application (and you have them passing), you should see:

```
$ python3 -m pytest -v -s test/test_q02.py
============================== test session starts ==============================
platform linux -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 -- /opt/base/bin/python3
cachedir: .pytest_cache
rootdir: /workspaces/assg03-solution
collected 5 items

test/test_q03.py::test_small_loan PASSED
test/test_q03.py::test_medium_loan PASSED
test/test_q03.py::test_large_loan PASSED
test/test_q03.py::test_zero_interest PASSED
test/test_q03.py::test_q03_application
Question <q03_application> Test 01:  PASSED
Question <q03_application> Test 02:  PASSED
Question <q03_application> Test 03:  PASSED
Question <q03_application> Test 04:  PASSED

 ========================================================================
 All attempted question tests passed  (004 passed of 004 tests attempted)
PASSED

============================== 5 passed in 0.10s ==============================
```

Here the `q03_application` tests are being invoked by the `pytest` module, but you can still run these by hand if you wish:

```
$ python3 test/test-question.py -c q03_application.py

Question <q03_application> Test 01:  PASSED
Question <q03_application> Test 02:  PASSED
Question <q03_application> Test 03:  PASSED
```

```
Question <q03_application> Test 04:  PASSED

 ============================================================================
 All attempted question tests passed  (004 passed of 004 tests attempted)
```

Question 1: Ackermann's Function (6 points)

Ackermann's Function is a recursive mathematical algorithm that can be used to test how well a system optimizes its performance of recursion. Design a function ackermann(m, n), which solves Ackermann's function. Parameters m and n are expected to be positive or 0 integers, and the function returns an integer result. Use the following logic in your function:

```
if m = 0 then return n + 1

if n = 0 then return ackermann(m - 1, 1)

otherwise return ackermann(m - 1, ackermann(m, n-1))
```

Question 2: Recursive Palindrome Checker (9 points)

For this question you are going to design two functions to test if a string is a palindrome or not. A palindrome is a string that that reads the same backward as forward. So for example "racecar" is a palindrome. We will consider case and space as significant in your tests, so "Racecar" should not be considered a palindrome for this questions, since the initial letter is a capital R and the final is a lower case R.

First create a Python solution taking advantage of the string library / string manipulations. Call this function `is_palindrome_iterative()` This function takes a string as input, and returns a boolean result True if the string is a palindrome, and false if it is not. The function should be implemented by using Python to reverse the given input string, then comparing the original string to its reverse to see if they are equal or not.

Next Design a function called `is_palindrome_recursive()` that again takes a string as input and returns a boolean result of True if the string is a palindrome, and false if not. However this time the function should use recursion to determine whether the word is a palindrome.

Hint: use string slicing to refer to and compare the characters on either end of the string. If the begin and end character are not the same, it cannot be a palindrome. Otherwise if they are the same character then the middle of the string has to be a palindrome for the whole string to be one. The base case(s) here are that a string that is of size 1 or size 0 is considered a palindrome by default.

Note: we use the same tests for both of your functions, but you will have points reduced if your first function is not implemented using Python string manipulations as described, or if you second function is not a recursive function with appropriate base and recursive cases defined.

Question 3: Mathematical Factorial and Combinations (15 points)

From basic probability, there is a concept known as counting the number of combinations of n items where you choose i items at a time. For example lets say you have $n = 4$ items **ABCD**, and we want to find all of the combinations where we choose $i = 2$ at a time. When counting combinations, order does not matter, so AB and BA are considered the same combination of 2 items chosen from the set of 4. From this set of 4 items you should be able to convince yourself that there are a total of 6 unique combinations: **AB**, **AC**, **AD**, **BC**, **BD**, **CD**.

Mathematically this is usually written like this:

$$\binom{4}{2} = 6$$

And it is read as the number of combinations of 4 itmes chosen 2 at a time yields 6 unique combinations.

One way to calculate the number of combinations of n items taken i at a time is by using the following formula

$$\binom{n}{i} = \frac{n!}{i!(n-1)!}$$

Where $n!$ is known as thew factorial function. The factorial of a value n is simply

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 1$$

For example

$$5! = 5 \times 4 \times 3 \times \times 2 \times 1 = 120$$

We could rewrite this as a recursive definition, e.g.

$$5! = 5 \times 4!$$

Write a recursive function called `factorial()` that takes a positive (or 0) integer n as input and returns the calculated factorial of n. This is required to be a recursive function, so you should call `factorial()` recursively to get the result for $n-1$, and multiply that by the original n to get the final result. For this function 1! is a base case, so if asked to compute the factorial of 1 you should just return 1. Also by definition 0! is considered to be 1. So you are required to implement 1! and 0! as base cases here that both return 1 as the result for this function.

Once you have factorial working, write a function named `combinations()` that takes n and i as input parameters, and returns the number of combinations of n items when i items are chosen at a time. This function is not recursive, but you should reuse and call your `factorial()` function to implement the counting of the number of combinations using the formula given above.

There is another way to calculate the number of combinations of n items where i are chosen. A recursive definition for this function is:

$$\binom{n}{i} = \binom{n-1}{i-1} + \binom{n-1}{i}$$

Notice that this is a recursive definition to calculate the number of combinations. We can recursively call our function 2 times, and add the result from the two smaller problems to get the answer to the original problem.

There are 2 base cases you need in order to implement stopping conditions for your recursive function. These are:

$$\binom{n}{0} = \binom{n}{n} = 1$$

If you think about it, this should make sense. The first one says there is only 1 way to chose 0 items from a set of any n items (e.g. don't choose any). The second one likewise says there is only 1 way to choose all n items from a set of n items.

Write another function named `combinations_recursive()`. This function will calculate the same thing as your previous one, to count the number of combinations from n items when i items are chosen, so it again takes the same two parameters n and i as input. However this function is required to be implemented recursively, using the recursive formula for combinations given above. You also need to detect the two base cases shown, when i is 0, or when $n == i$, and return 1 as the answer in those base cases.

Assignment Conclusion / Checklist

Hopefully you have created commits after completing each question and pushed them successfully to your assignment repository. Make sure that you do the following for this and all class assignments.

- Check your autograder results after pushing your commits. Look in your `Feedback` pull request, and in the detailed autograder report generated for your release(s) made for each commit.

- You should never close or merge your `Feedback` pull request, as is mentioned in the comment. The instructor will evaluate your assignments, and may give you feedback about your code or work. Check here after the assignment is returned for a code review and assignment comments.
- In general this class will ask you to follow and use good Python style as specified by the official Python PEP 8 Style guide. You should at least pay attention to
  - All indentation is required to be 4 spaces with no tabs in files.
  - Maximum line lengths should be usually observed, do not generally extend code past the 79 character column in a file.
  - Follow the suggestions for breaking long lines in code (e.g. line break before binary operators in a long expressions).
  - Prefer single quotes for all string in code for class assignments, unless you need a string with a single quote, for example to add an apostrophe 's.
  - Follow Pep8 style for whitespace in expressions and statements.
    For example, usually a single space should go before and after all binary operators in expressions.
  - We encourage the use of function annotations in Python code.
- Also in general in this class we will ask you to follow Git Commit Best Practices
- Strive to use Meaningful Names for variables, constants and functions in your code. You are required to follow Pep8 naming guidelines, which means `snake_case` names for variables and functions and `SCREAMING_SNAKE_CASE` for constants. Python style switches to `PascalCase` for class names.

Make sure you are checking review comments and code review given for your class assignments. We may start by making suggestions where your style or practices could be improved. As the class progresses, some of these suggestions may become requirements, especially for students who are repeatedly making the same style or practice error and are not following feedback to correct a noted issue in future assignments. Assignments may be left ungraded in some cases for style or practice issues until corrected, and/or may have points removed or receive a 0 grade if issues continue after receiving multiple feedback that are repeatedly ignored and not corrected.

Additional Information

The following are suggested online materials you may use to help you understand the tools and topics we have introduced in this assignment.

- Python PEP 8 Style guide
- Git Commit Best Practices
- Git Best Practices
- Best Practices to Write Readable and Maintainable Code: Choosing meaningful names
- How to Write Better Names for your Variables, Functions and Classes